

A Hybrid Approach of Imitation Learning and Deep Reinforcement Learning with Direct-Effect Update Interval for Elevator Dispatching

JIANSONG WAN, KANGHOON LEE, and HAYONG SHIN, Korea Advanced Institute of Science and Technology, Daejeon, Republic of Korea

The rapid increase in high-rise building construction has intensified the need for efficient elevator system operations. This article addresses the elevator dispatching problem in elevator group control systems. We formulate the problem as a Semi-Markov Decision Process (SMDP), defining the state representation, action space, and reward function. A two-phase model is then introduced, integrating imitation learning and deep reinforcement learning techniques to derive the optimal elevator dispatching policy from the formulated SMDP. In the first phase, a policy network is pre-trained by estimating the time required for elevator cars to pick up assigned hall requests. In the second phase, the pre-trained policy network is further optimized using Proximal Policy Optimization (PPO), a well-known policy-based deep reinforcement learning method. Additionally, we propose a novel update interval, termed the “direct-effect” interval, which improves policy training during the reinforcement learning phase. Notably, this direct-effect interval concept has potential applicability to other multi-resource scheduling problems. Empirical experiments demonstrate the advantages of incorporating imitation learning before reinforcement learning, as well as the effectiveness of employing the direct-effect update interval during the reinforcement learning phase. Furthermore, the proposed model outperforms various benchmark rules in terms of average waiting time and the distribution of long waiting times, as validated across four traffic patterns.

CCS Concepts: • **Computing methodologies** → **Reinforcement learning**;

Additional Key Words and Phrases: Elevator dispatching, group control, semi-Markov decision process, imitation learning, deep reinforcement learning

ACM Reference format:

Jiansong Wan, Kanghoon Lee, and Hayong Shin. 2026. A Hybrid Approach of Imitation Learning and Deep Reinforcement Learning with Direct-Effect Update Interval for Elevator Dispatching. *ACM Trans. Cyber-Phys. Syst.* 10, 3, Article 30 (May 2026), 19 pages.
<https://doi.org/10.1145/3790252>

This work was partially supported by the National Research Foundation of Korea (NRF) through a grant funded by the Korean government (Ministry of Science and ICT, MSIT) (No. RS-2022-NR068758).

Authors' Contact Information: Jiansong Wan (corresponding author), Korea Advanced Institute of Science and Technology, Daejeon, Republic of Korea; e-mail: jswan@kaist.ac.kr; Kanghoon Lee, Korea Advanced Institute of Science and Technology, Daejeon, Republic of Korea; e-mail: leehoon@kaist.ac.kr; Hayong Shin, Korea Advanced Institute of Science and Technology, Daejeon, Republic of Korea; e-mail: hyshin@kaist.ac.kr.



This work is licensed under Creative Commons Attribution International 4.0.

© 2026 Copyright held by the owner/author(s).

ACM 2378-9638/2026/5-ART30

<https://doi.org/10.1145/3790252>

1 Introduction

The performance of elevator systems largely depends on the **Elevator Group Control System (EGCS)**, which manages multiple elevator cars to satisfy transportation demands while optimizing key performance indicators such as average waiting time, long-wait percentage, and energy consumption [35]. Among EGCS functions—such as data collection and traffic pattern recognition—elevator dispatching is the most critical, as it directly determines service efficiency and passenger experience. Elevator dispatching refers to the assignment of passenger-initiated hall calls to elevator cars. In conventional systems, passengers press up/down buttons, and the EGCS assigns a car to serve the request. Advanced technologies such as destination control systems, double-deck elevators, and multi-car systems have been introduced to improve efficiency [6, 16, 29].

Finding an optimal dispatching policy is well-known to be NP-hard [8, 15]. Traditional methods—such as heuristic rules [20], fuzzy logic control [21], and evolutionary algorithms [5, 6]—offer feasible but suboptimal solutions due to the stochastic and dynamic nature of elevator systems. Exact mathematical formulations [22] and search-based approaches [8] provide theoretical insight but rely on strong assumptions or precise cost estimations that are often impractical in real-time operations. Recent studies have introduced deep learning to extract useful contextual information—such as elevator occupancy [33] and passenger arrival patterns [35]—to support better dispatching decisions. However, these approaches typically separate feature extraction from decision-making, limiting their ability to jointly optimize information use and control performance.

Reinforcement Learning (RL) provides a promising framework to enable self-learning elevator systems that improve policies through interaction [7, 34]. Earlier RL-based dispatching models demonstrated performance gains [13, 14, 34], but they suffer from two major limitations. Firstly, they tend to solely focus on training the agent from scratch, overlooking the potential of leveraging well-performing dispatching rules as valuable “teachers” for the agent to imitate. Training an RL agent requires an extensive collection of data samples obtained through agent-environment interactions. Training the agent entirely from scratch using RL can be costly in terms of sample efficiency and effectiveness. In contrast, **Imitation Learning (IL)** does not rely on trial-and-error exploration. If expert demonstrations are of sufficient quality, IL enables the agent to quickly approximate a policy close to expert performance. RL can then be applied to refine and improve this pre-trained policy. Secondly, existing RL-based elevator dispatching studies often adopt the transition interval between two decision events (referred to as the “decision-event” interval throughout the rest of this article) as the update interval for RL agent training. However, in elevator systems, a time gap exists between assigning a hall call to a specific elevator car and the elevator car picking up the hall call. Merely using the “decision-event” interval fails to accurately generate reward signals and does not reflect the full impact of chosen actions for the RL agent, compromising the effectiveness of the RL training. As illustrated in Figure 1, the events associated with each floor and direction are listed—for example, Floor 2 (Up) and Floor 3 (Down). All discrete events can be merged into a single timeline, allowing us to observe the variations across different time intervals.

To address these limitations, this article proposes a hybrid IL and deep RL framework for elevator dispatching. Our model formulates the problem as a **Semi-Markov Decision Process (SMDP)**, which is more suitable for modeling elevator systems than the traditional **Markov Decision Process (MDP)** used in most existing RL-based elevator dispatching studies. The continuous nature of the time interval between two discrete events, such as the release of new hall requests, is more appropriately handled by SMDP. The formulated SMDP is solved in two phases: the imitate phase and the surpass phase. In the imitate phase, a policy network is pre-trained by estimating the time required for elevator cars to pick up assigned hall calls. This pre-trained policy network is then further refined using **Proximal Policy Optimization (PPO)**, a well-known policy-based deep

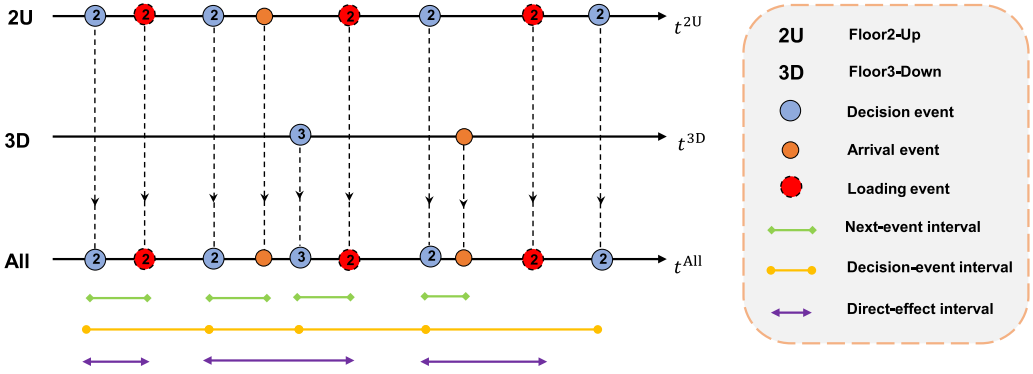


Fig. 1. Illustration of various update intervals: *Next-event interval* often triggers early updates, which may not fully capture the impact of taken actions. *Decision-event interval*, on the other hand, consistently leads to either early or late updates. The proposed *direct-effect interval* ensures timely updates that accurately reflect the influence of taken actions.

RL technique. To improve the efficiency of RL training, we propose using the interval between the agent selecting an action and the completion of that action—referred to as the “direct-effect interval”—for policy updates, instead of the commonly used decision-event interval in prior studies. The decision-event interval often results in inaccurate advantage estimation due to misaligned reward signals. In contrast, the direct-effect interval ensures that policy updates occur after the full impact of the action is observed, enabling more accurate and lower-variance advantage estimates. We empirically demonstrate the benefit of incorporating IL prior to the RL phase in terms of sample efficiency and effectiveness. Furthermore, we validate the superiority of the “direct-effect” interval during the RL phase. Finally, we verify the superiority of our model over popular benchmark rules in elevator studies regarding average waiting time and the percentage of long waiting times under various traffic patterns.

The remainder of the article is organized as follows. Section 2 reviews related work. Section 3 formulates the elevator dispatching problem as an SMDP. Section 4 details the proposed hybrid imitation–RL framework. Section 5 presents experimental results and analysis, and Section 6 concludes the article with future directions.

2 Literature Review

This section introduces the elevator dispatching problem and its relation to other dispatching tasks, followed by a brief overview of IL and RL foundations. Finally, we review prior studies that have applied RL to elevator dispatching.

2.1 Elevator Dispatching Problem

The elevator dispatching problem—also known as the car-landing call assignment problem—aims to determine the optimal matching between hall calls and elevator cars. This problem is NP-hard, as the search space grows exponentially with the number of hall calls while the number of cars remains fixed. Real-world elevator systems add further complexity due to uncertainty and incomplete information. The number of passengers behind a hall call and their destinations are unknown until boarding occurs. Moreover, dispatching decisions must balance multiple, often conflicting objectives, such as minimizing waiting time while reducing energy consumption [18]. Existing approaches include heuristic rules, fuzzy logic control, evolutionary algorithms, and mathematical programming, as summarized in Table 1. Recently, RL has gained attention for its ability to learn

Table 1. Summary of Major Methods for Elevator Dispatching

Method	Pros	Cons
Heuristic rules [20]	Simple; interpretable	Poor adaptability to dynamics
Evolutionary algorithms [5, 6, 27]	Good global search	Slow and requires cost modeling
Graph optimization [8, 19, 22]	High-quality solutions	Needs full info; sensitive to cost accuracy
DL + separate model [33, 35]	Learns useful features	Weak integration with decision-making
End-to-end DRL [7, 14, 34]	Adaptive and self-learning	Complex formulation and data-intensive

adaptive, data-driven policies without explicit system modeling. This study extends the application of RL in elevator dispatching by addressing its training inefficiency and delayed reward issues.

It is worth noting that the elevator dispatching problem studied in this article shares similarities with other related vehicle dispatching problems, such as the Overhead Hoist Transport dispatching problem [31], **Automatic-Guided Vehicles (AGV)** scheduling problem [11], and the taxi dispatching problem [17]. These problems also aim to find the optimal assignment decisions between transportation requests and moving instruments. However, two main points differentiate the elevator dispatching problem from the vehicle dispatching problem. First, elevator dispatching allows an elevator car to be assigned to multiple requests, making it a multi-load problem. Second, the elevator dispatching problem exhibits distinct patterns of transportation requests that vary throughout the day. According to classic elevator traffic theory [3], there are four traffic patterns: up peak, inter-floor, lunch peak, and down peak. In fact, the elevator dispatching problem has received relatively less attention compared to the aforementioned vehicle dispatching problems.

2.2 IL and RL

IL is a paradigm in which an agent learns to replicate expert behavior through demonstrations [12, 36]. IL methods are commonly categorized into three major approaches: **Behavior Cloning (BC)**, which learns a direct mapping from system states to expert actions [28]; **Inverse RL (IRL)**, which attempts to recover the reward function from demonstrations to guide action selection [1, 2]; and **Generative Adversarial IL**, in which a generator iteratively optimizes and outputs a policy, while a discriminator distinguishes between generated and expert policies [10].

Among these, BC is the simplest and most efficient, framing imitation as a classification or regression problem depending on the action space. However, its performance is bound by that of the expert demonstrations. Combining IL with RL allows the pre-trained policy to be further refined through trial-and-error interaction, often surpassing expert-level performance. RL enables an agent to learn an optimal policy by interacting with an environment to maximize cumulative rewards [26]. With the integration of deep neural networks, **Deep RL (DRL)** has achieved notable success across domains such as traffic control, logistics, and energy management [4, 30, 32].

Most representative DRL algorithms are model-free, where the agent does not learn about the environment dynamics. The model-free DRL can further be divided into value-based and policy-based methods. The value-based methods usually learn a state-action value function $Q(s, a; \theta)$, and then the agent selects actions that maximize the estimated Q values. On the other hand, the policy-based methods directly optimize a policy $\pi(a|s; \theta)$ by updating it toward the direction of maximizing the cumulative reward that the agent may get.

The policy-based methods may also learn a value function in addition to the policy function learning, known as the actor-critic methods. The actor (a.k.a. policy function) returns a distribution over actions given the state. In contrast, the critic (a.k.a. value function) evaluates actions taken by the actor by providing the estimate of the return the agent may get. For most actor-critic methods,

the gradient estimator G for policy updating can be summarized in one equation:

$$G = \mathbb{E} [\nabla_{\theta} \log \pi(a | s; \theta) \hat{A}],$$

where $\pi(a | s; \theta)$ is the parameterized stochastic policy to optimize, and $\hat{A}$ is an estimator of the advantage function. In the Advantage Actor Critic method, $\hat{A}$ is the one-step **Temporal Difference (TD)** advantage calculated through another parameterized state value function.

To improve training stability, Trust Region Policy Optimization constrains policy updates within a safe region [23]. The more efficient PPO algorithm later replaced the constraint with a clipped surrogate objective [25], avoiding complex second-order optimization while maintaining stable learning performance.

2.3 RL for Elevator Dispatching

The elevator dispatching problem has been addressed using RL in a multi-agent view in [7], where each elevator car was treated as an independent agent. They employed the Q-learning algorithm to learn the action-value function, approximated by a fully connected neural network to overcome the “curse of dimensionality.” However, their use of a simple neural network and Q-learning algorithm leaves room for improvement, particularly considering the recent advancements in deep RL techniques. Since then, researchers have continued to explore the potential of RL in solving the elevator dispatching problem. For instance, RL agents that adaptively selected dispatching rules from a pool of heuristic rules based on dynamically changing system situations were developed in [14]. A similar approach in the context of a multi-car elevator system was investigated [13], where multiple elevator cars operate within a single shaft. It is worth noting that this kind of approach has a potential limitation as the final performance heavily relies on the design of the heuristic rule pool. Suboptimal pool design may significantly restrict the agent’s ability to achieve optimal performance. In a recent work [34], deep convolutional networks and the modern Asynchronous Advantage Actor Critic algorithm were used as the foundation of their model. The powerful feature extraction capabilities of deep neural networks enhanced the agent’s understanding of the complex and dynamic state of the elevator system, thereby improving the overall system performance. However, their work assumes the availability of perfect information, such as the number of people waiting behind the hall call, which is often difficult to access in conventional elevator systems. This assumption limits the practicality of their approach for real-world implementation.

3 SMDP Formulation

The elevator dispatching problem involves decision-making that affects the availability of elevator cars in the future. Additionally, passengers continuously enter the elevator system and make requests from the hall. These aspects transform the elevator dispatching problem into a sequential decision-making problem with a potentially infinite optimization horizon. Consequently, to make optimal assignment decisions, it is essential to adopt a farsighted approach that considers the impact of present decisions on future outcomes. In this regard, we address the elevator dispatching problem by formulating it as an SMDP and propose a two-phase model to effectively solve this formulated SMDP. Unlike a discrete-time MDP, an SMDP is a continuous-time generalization where the duration of time spent in each state is modeled as a general real-valued variable. Given that the time intervals between discrete events are real-valued variables and the passenger arrival rates are also dynamically changing, SMDP is more suitable for our purposes compared to an MDP. Empirical experiments and discussions comparing the SMDP and MDP formulations for the elevator dispatching problem are presented in Appendix A. In the subsequent subsections, we provide detailed explanations of the state representation, action space, and reward function in our SMDP formulation.

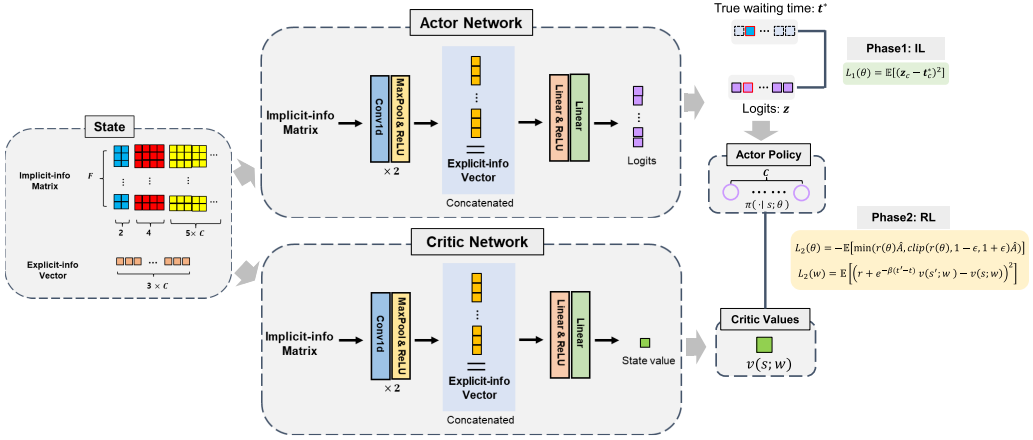


Fig. 2. Overview of the proposed model.

3.1 State Representation

A high-quality state representation plays a crucial role in enabling the agent to obtain an accurate understanding of the system's situation, thereby enhancing its decision-making capabilities. The state representation in our formulation is depicted on the left side of Figure 2. It incorporates two types of information: implicit information in matrix form and explicit information in vector form. Suppose the number of elevator cars is C . The implicit information matrix contains details requiring deeper knowledge extraction. It is represented as a matrix with dimensions of $F \times (2 + 4 + 5 \times C)$, including current request identification (blue, two columns), hall information (red, four columns), and car information (yellow, $5 \times C$ columns). Each element in row f and column k can be regarded as the k th feature of floor f . The followings describe each feature of floor f :

- $k = 1$ (2): binary number, indicating whether the current request at floor f is to go up (down);
- $k = 3$ (4): binary number, indicating whether the up (down) hall button is pressed at floor f ;
- $k = 5$ (6): real number, elapsed time since up (down) hall button is pressed at floor f ;
- $k = 6 + 3 \times j + 1$: binary number, indicating whether passengers in car j will get off at floor f ;
- $k = 6 + 3 \times j + 2$ (3): binary number, indicating whether car j is at floor f with a moving up (down) state;
- $k = 6 + 3 \times j + 4$ (5): binary number, indicating whether the up (down) request at floor f has been assigned to car j .

The explicit information provides specific details about the car that can be easily processed. It consists of three types of information: the number of passengers in the car, the remaining capacity of the car, and the expected time required for picking up a new request. We combine this explicit information with the processed implicit information.

Our state formulation reflects the characteristics of conventional up-down elevator systems, in which exact passenger counts are generally unobtainable. A hall button press signals a request but does not reveal how many passengers are waiting, and a car button press only indicates that at least one passenger intends to alight on that floor. In contrast, perfect information—such as the exact number of passengers disembarking on each floor or waiting behind a hall call—is typically available only in advanced destination control systems.

With the increasing adoption of Internet of Things technologies, however, such detailed information is becoming more accessible even in traditional up-down systems through the use of

additional sensors and cameras. Several studies have therefore assumed the availability of perfect information. In our work, we adopt the more general and widely applicable assumption that this information is not available. Nevertheless, to assess its potential value, we also investigate how providing perfect information affects system performance. Readers may refer to Experiment 4 for a detailed discussion.

3.2 Action Space

The action space represents all possible actions that the agent can perform at decision-making moments. In our formulation, the number of feasible actions at each decision moment corresponds to the total number of elevator cars, denoted as C . That is to say, when passengers generate a new hall call request, the agent is required to select one of the available elevator cars to fulfill the request.

3.3 Reward Function

The reward function is designed to evaluate the agent's action selection and optimize decision-making. In our formulation, the reward is calculated by accumulating partial rewards within a defined update interval. Each partial reward is the discounted integral over the instantaneous reward r_τ at each continuous time moment τ between two successive discrete events.

Let us consider the "direct-effect" interval, which starts at a decision-event t and ends at a pick-up event t' . Additionally, suppose there are other n discrete events that occurred at $t_1, t_2, \dots, t_n$ during the interval. The reward for this transition interval can be calculated as follows:

$$R(s, a) = \sum_{(t^l, t^c) \in \{(t, t_1), (t_1, t_2), \dots, (t_n, t')\}} \underbrace{\int_{t^l}^{t^c} e^{-\beta(\tau-t)} r_\tau d\tau}_{\text{Partial reward}}. \quad (1)$$

The key is how to define r_τ . Here, we analyze three alternative r_τ functions with different exponents:

$$r_\tau = - \underbrace{\sum_{p \in \mathcal{P}^{hall}} \left(\tau - t_p^{arrive} \right)^i}_{\text{waiting time signal}}, \quad (2)$$

where i can take the values 0, 1, or 2.

For the three exponents, when $i = 0$, each passenger p in the hall passenger set $\mathcal{P}^{hall}$ or the car passenger set $\mathcal{P}^{call}$ yields an instantaneous reward of -1 , regardless of their arrival time t_p^{arrive} or boarding time t_p^{board} . With $i = 1$, the instantaneous reward linearly relates to the passenger's waiting time at the hall or in the car. For $i = 2$, the instantaneous waiting time reward decreases quadratically as the waiting time at the hall increases. Thus, passengers with longer waiting times produce less reward, reflecting the agent's sense of urgency. We choose $i = 1$ for the instantaneous reward definition in the experiment section to consider the consistency with minimizing average waiting time as well as reducing long waiting times for fair service. Moreover, to ensure that the reward signal is a quantity we want to maximize, we introduce a minus sign.

4 Model Architecture and Training

In this section, we present a comprehensive explanation of the network design used in our model. We begin by describing the architectural choices and configurations of the networks. Then we elaborate on the two-phase training process based on IL and deep RL.

4.1 Model Architecture

Figure 2 provides an overview of the proposed model. Firstly, the implicit information matrix is passed through a convolutional block comprising two one-dimensional convolution layers. The one-dimensional convolution operation is performed along the floor direction, enabling the extraction and preservation of information from all floors within the block. The output of the convolutional block is then flattened into a one-dimensional vector, which is subsequently concatenated with the explicit information vector. Finally, the concatenated vector is used as input to a Multi-Layer Perceptron to generate the final logits for the actor network or the state value for the critic network. Our model employs separate actor and critic networks without parameter sharing to prevent interference between the objectives of the two networks.

4.2 Phase 1: Pre-Train via IL

In this phase, we pre-train the policy network by estimating the time required for elevator cars to respond to assigned hall calls, as illustrated in Figure 2. To achieve this, we conduct simulations using the **Expected Time of Arrival (ETA)** rule-based dispatching policy to collect data points. Whenever a new hall call occurs, we record the system state s at that moment along with the assigned car c for handling the hall call based on the ETA rule. Subsequently, once the hall call has been picked up, we record the actual time taken for the assigned car to handle this hall call. It is important to note that we can obtain the actual pick-up time only for the assigned car. However, the policy network provides estimated pick-up times for all elevator cars. Hence, when calculating the loss during training using the collected dataset, we need to mask the other cars and solely consider the assigned car.

Once we have gathered a sufficient amount of data, we train the policy on the collected dataset D in a supervised manner, with the objective of minimizing the mean squared error between the estimated pick-up time for the assigned car c , denoted as z_c , and the actual pick-up time denoted as t_c^* . Essentially, this actually equals solving a regression problem:

$$\mathcal{L}_1(\theta) = \mathbb{E}_{(s,c,t_c^*) \sim D} \left[(z_c - t_c^*)^2 \right]. \quad (3)$$

We have chosen to pre-train the network by imitating the actual pick-up times, rather than relying solely on the ETA decision, for two main reasons. Firstly, because we incorporate the estimated pick-up time obtained from the ETA into the state representation of our SMDP formulation, if we were to pre-train the policy by imitating the ETA decision alone, the policy network might rely heavily on the ETA feature and neglect the appropriate adjustment of weights for other relevant features. Secondly, imitating the true pick-up times has the potential to yield a superior policy compared to relying solely on ETA. The true pick-up time provides a more accurate and reliable basis for dispatching decision-making.

4.3 Phase 2: Improve via RL

In this phase, our goal is to enhance the pre-trained policy using RL. Specifically, we utilize the PPO algorithm, which belongs to the family of actor-critic frameworks. The actor loss function, denoted as $\mathcal{L}_2(\theta)$, is a negative clipped surrogate objective defined as follows:

$$\mathcal{L}_2(\theta) = \mathbb{E} \left[-\min \left(r(\theta) \hat{A}, \text{clip} \left(r(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A} \right) \right]. \quad (4)$$

Here, $r(\theta) = \frac{\pi(a|s;\theta_{\text{new}})}{\pi(a|s;\theta_{\text{old}})}$ represents the ratio of probabilities between the updated policy and the previous policy, and ϵ is a small constant that restricts the policy update within certain trust region.

Table 2. Simulator Setups

System component	Settings
Building	Number of floor: 20 Floor height: 3 m Population: 1,200
Car	Number of cars: 4 Full speed: 3 m/s Acceleration: 1.5 m/s ² Capacity: 20 $t_{0,0}$: $2\sqrt{2}$ s $t_{0,F}$: 2 s $t_{F,0}$: 2 s $t_{F,F}$: 1 s

The one-step TD advantage, denoted as $\hat{A}$, is defined as follows:

$$\hat{A} = R(s, a) + e^{-\beta(t'-t)} v(s'; w) - v(s; w). \quad (5)$$

In this equation, $e^{-\beta(t'-t)}$ serves as an adaptive discount factor, which differs from the constant discount factor used in traditional MDP formulations. In our SMDP formulation, the discounting effect is determined by the continuous time interval $t' - t$ the transition spans.

The critic function $v(s; w)$ is updated by minimizing the following loss function, denoted as $\mathcal{L}_2(w)$:

$$\mathcal{L}_2(w) = \mathbb{E} \left[\left(R(s, a) + e^{-\beta(t'-t)} v(s'; w) - v(s; w) \right)^2 \right]. \quad (6)$$

In summary, the PPO algorithm updates the policy network by minimizing the actor loss $\mathcal{L}_2(\theta)$ while simultaneously updating the critic network by minimizing the critic loss $\mathcal{L}_2(w)$. This dual optimization process enables our model to converge to an improved and more efficient elevator dispatching policy.

5 Numerical Results

This section describes the setups for our experiment and a detailed analysis of the numerical results. The simulator environment and the proposed model are both implemented in Python.

5.1 Simulator Setups

The experimental environment consists of a 20-floor building, with each floor having a height of three meters. The system comprises four identical elevator cars, each capable of accommodating up to 20 people and operating at a maximum speed of 3 m/s. Our simulator incorporates the effects of acceleration and deceleration by assuming that the travel times between floors vary based on different movement scenarios. Specifically, we assume that the elevator car uniformly accelerates from rest to full speed (and decelerates from full speed to rest) over the distance of one floor height. This enables us to calculate the acceleration and travel times for various movement scenarios. For example, $t_{0,0} = 2\sqrt{2}$ s indicates that it takes $2\sqrt{2}$ s for the elevator car to start moving from a stopped position at a floor and reach the next floor. The travel time for an elevator car passing a floor at full speed and stopping at the next floor is denoted as $t_{F,0} = 2$ seconds. Similarly, if an elevator car passes the current floor at full speed and continues to pass the next floor, the travel time is 1 seconds, represented as $t_{F,F} = 1$ seconds. The setup details of the simulator are summarized in Table 2.

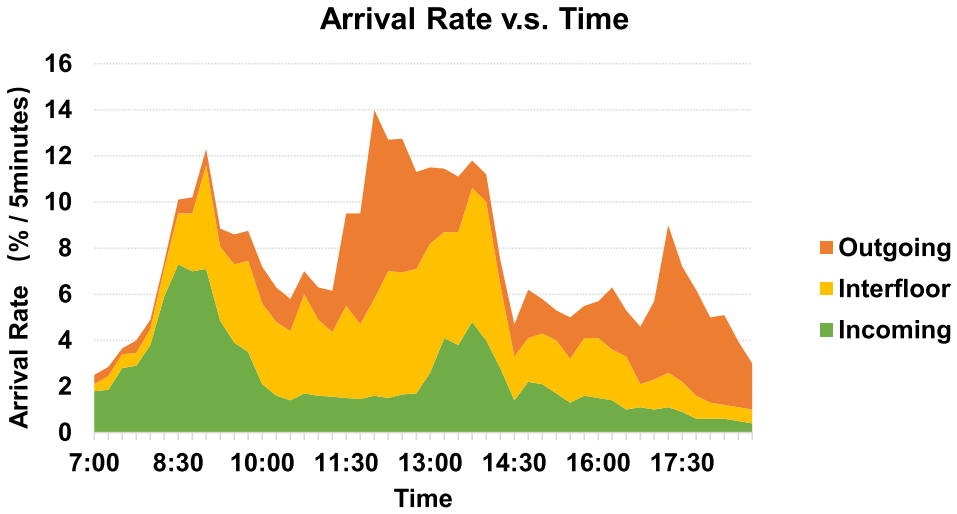


Fig. 3. The daily traffic profile is used for experiments. The arrival rate of passengers in a 5-minute period is given as a percentage of the total population of the building.

The daily traffic profile utilized in our simulator is adapted from the work of [9]. As depicted in Figure 3, this profile captures the dynamic traffic patterns observed in an office building from 7:00 to 19:00. These patterns correspond to the four well-known traffic patterns in elevator traffic theory: up peak (7:00–10:00), inter-floor (10:00–11:30, 14:30–16:00), lunch peak (11:30–14:30), and down peak (16:00–19:00). The profile encompasses three types of traffic flow: incoming (passengers traveling from the lobby to other floors), outgoing (passengers traveling from other floors to the lobby), and inter-floor (passengers traveling between floors in both directions). For instance, during the lunch peak, the outgoing traffic flow initially rises and then declines as many passengers leave for lunch or depart for home. Subsequently, the incoming flow increases as these individuals return to their workplace.

To compare the performance of our proposed model, we also implement five benchmark rules that are widely used for comparison in elevator dispatching studies. The first rule, **Round Robin (RR)**, assigns requests to elevator cars in a circular order. The “scan” and “look” rules involve elevator cars continuously shuttling between the lobby and the top floor, but with a slight distinction. The “look” rule allows the elevator car to change direction if there are no further requests in its current direction. The **Genetic Algorithm (GA)**-based dispatching rule uses GA to find the dispatching decision with the highest fitness. The definition of fitness function and the gene follows the work [27]. Lastly, the ETA rule [20] estimates the expected time for each car to pick up a request and assigns the request to the car with the earliest arrival time.

5.2 Result Analysis

Through the experimental evaluation, we aim to answer the following questions:

- *Effect of IL phase*: How does the IL phase benefit the training of our model?
- *Comparison of various update intervals*: How does the proposed “direct-effect” interval compare against other update intervals?
- *Comparison with benchmark rules*: How does our model compare against heuristic rules that are widely used as benchmarks in previous works?

- *Effect of using perfect information*: How does the perfect information benefit the performance of our model?

The general experimental setups for our study are as follows: we collect data over a span of 60 simulation days for the up peak pattern and 30 simulation days for each of the other three traffic patterns. We recommend collecting additional data for the up peak pattern due to the relatively lower number of data points obtained compared to other patterns. This occurs because we consider the new request as the decision event; during the up peak, many passengers wait on the lobby floor (not releasing new requests if the hall button is pressed). However, in other traffic patterns, passengers arrive more evenly on other floors, releasing more requests and triggering more decision events.

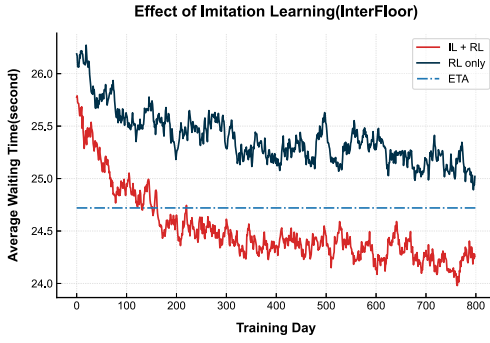
After obtaining the demonstration data, we pre-train the policy network using the collected samples. The pre-trained network is then further refined through RL over 800 simulation days. Our experiments are conducted on a single GPU-equipped machine (NVIDIA GeForce RTX 2080 Ti), and training for 800 simulated days under a single traffic pattern takes approximately 20 hours. Therefore, training across all four traffic patterns can be completed in less than four days. To ensure reliable and statistically significant results, we evaluate the trained agent five times using different random seeds. The model hyperparameters are carefully selected by experimenting with multiple configurations. The effects of several representative hyperparameters are presented in Appendix B.

5.2.1 Effect of Imitating Learning Phase. In this experiment, our objective is to investigate the impact of incorporating IL before the RL phase. We compare the training processes of our model under two scenarios: training the RL agent from scratch (RL-only agent) and using IL followed by RL (IL + RL agent). Specifically, we focus on the interfloor and down peak patterns. We encountered difficulties in successfully training an RL agent without the inclusion of IL for the lunch peak and up peak patterns. As previously mentioned, the initialized policy performed even worse than a random policy, resulting in the agent consistently encountering catastrophic states during its interactions with the environment.

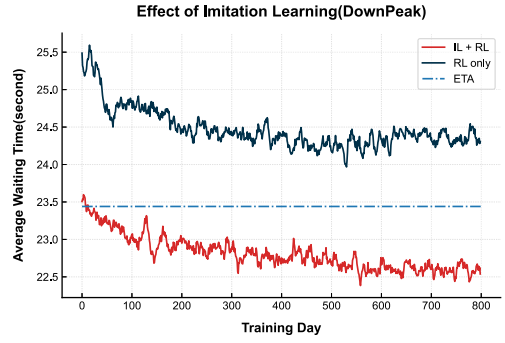
Figure 4 illustrates a comparison of the training processes. To ensure a fair comparison, we trained the RL-only agent for 850 days, taking into account the additional time spent on IL by the IL + RL agent (e.g., data collection and training). We omitted the training process of the RL-only agent for the first 50 days, which can be considered as its warming-up time. Despite this adjustment, the IL + RL agent exhibits a notably stronger starting point and more rapid performance improvement, highlighting the higher data efficiency of IL. If the expert demonstrations are of sufficiently high quality, supervised learning via IL allows the agent to quickly approximate a near-expert policy. This pre-trained policy not only ensures more stable behavior during interaction with the environment but also results in high-quality experience data. These advantages contribute to the superior sample efficiency and overall effectiveness of IL + RL compared to RL alone. In summary, even when both agents undergo the same RL training duration, the IL + RL agent consistently converges to a higher performance level. This result demonstrates the practical value of incorporating IL as a precursor to RL, effectively combining IL's data efficiency with RL's optimization capability to achieve superior performance.

5.2.2 Comparison of Various Update Intervals. This experiment aims to compare the effectiveness of our proposed “direct-effect” interval with other update intervals. In addition to the previously mentioned next-event interval and the decision-event interval, we also included the **Generalized Advantage Estimation (GAE)** interval [24]. It can be considered an improved version of the decision-event interval that balances different ranges of reward signals.

As depicted in Figure 5, all update intervals can be utilized for training RL agents, but they differ in terms of speed and final performance, indicating varying levels of effectiveness. The next-event

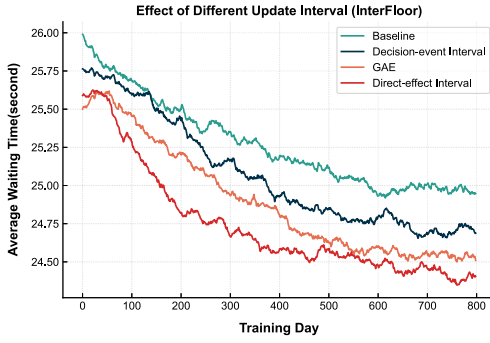


(a) InterFloor

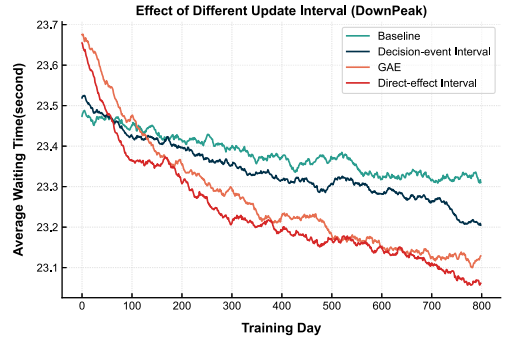


(b) DownPeak

Fig. 4. Comparison of training processes using RL only and using IL + RL.



(a) InterFloor



(b) DownPeak

Fig. 5. Comparison of different update intervals.

interval, as mentioned earlier, often triggers early updates, which may not fully capture the impact of the actions taken. On the other hand, the decision-event interval consistently leads to either early or late updates, failing to generate accurate reward signals for the RL agent. Although GAE attempts to address the limitations of the decision-event interval by considering both short-term and long-term rewards. However, it still falls short of fully resolving the associated issues. Notably, our proposed “direct-effect interval” exhibits superior performance in terms of both training speed and final performance achieved. To elaborate, the advantage function in Equation (5) quantifies the relative benefit of taking a specific action compared to the baseline. It is therefore crucial that the full effect of an action is accurately captured. If the immediate reward does not reflect the action’s true impact, the advantage estimate must rely more heavily on the value function, which introduces bias and variance due to the imperfections of the function approximator. The direct-effect update interval addresses this by aligning policy updates with the moments when an action’s impact is fully realized in the environment—particularly at task-critical transitions (e.g., passenger pickup or drop-off). This alignment results in more accurate and lower-variance advantage estimates, thereby producing more reliable policy gradient updates.

Table 3. Average Waiting Time Comparison under Four Traffic Patterns

		UpPeak	InterFloor	LunchPeak	DownPeak
Benchmark rules	RR	—	44.53 $\pm$ 0.95	—	36.65 $\pm$ 0.64
	Scan	220.93 $\pm$ 9.76	33.99 $\pm$ 0.41	141.01 $\pm$ 9.24	33.95 $\pm$ 0.22
	Look	170.54 $\pm$ 17.05	30.34 $\pm$ 0.30	136.48 $\pm$ 8.24	30.45 $\pm$ 0.31
	GA	115.73 $\pm$ 4.87	24.68 $\pm$ 0.33	120.12 $\pm$ 6.42	23.58 $\pm$ 0.24
	ETA	95.36 $\pm$ 5.04	24.72 $\pm$ 0.32	98.00 $\pm$ 6.40	23.44 $\pm$ 0.32
Ours (w.o. perfect info)	IL	101.48 $\pm$ 6.08	25.52 $\pm$ 0.38	103.85 $\pm$ 5.71	23.56 $\pm$ 0.35
	IL + RL	89.18 $\pm$ 8.07	23.95 $\pm$ 0.22	92.22 $\pm$ 6.06	22.51 $\pm$ 0.26
Ours (w. perfect info)	IL	98.03 $\pm$ 7.35	25.24 $\pm$ 0.07	99.29 $\pm$ 4.20	23.29 $\pm$ 0.34
	IL + RL	85.37 $\pm$ 6.68	23.78 $\pm$ 0.08	88.63 $\pm$ 2.65	22.40 $\pm$ 0.19

—, unmeaningful result because of overloading.

5.2.3 Comparison with Benchmark Rules. In this experiment, we conducted a comparison between our model and benchmark rules regarding the average waiting time and the percentage of long waiting times under four typical traffic patterns. The results for average waiting time are summarized in Table 3. Among the benchmark rules, the ETA rule performs the best. The imitating learning phase leads to a pre-trained policy with moderate performance, slightly inferior to ETA. It is important to note that theoretically, the pre-trained policy has the potential to outperform the ETA rule, as it directly imitates the true pick-up time rather than imitating the ETA decision. However, achieving this superior performance may be challenging due to the difficulty of accurately estimating the true pick-up time, especially when crucial information such as the number of passengers behind the hall call and the number of passengers getting off from a car is not provided. These factors significantly influence the true pick-up time. Nevertheless, after refining the pre-trained policy using RL, we were able to obtain a policy that outperforms the ETA rule across all four traffic patterns.

The long waiting time is also an important evaluation metric for assessing elevator system performance. We compared the waiting time percentiles of our proposed model and the ETA rule under the four traffic patterns, as shown in Table 4. The statistics are based on the 5-day test data of our proposed model and the ETA rule, recording each passenger's waiting time. Our model surpasses the ETA rule in terms of the percentage of passengers experiencing waiting times greater than 60 seconds (usually considered as the dissatisfaction threshold). Moreover, our model achieved shorter waiting times for the 0.95 and 0.75 percentiles under all four traffic patterns, demonstrating its superiority over the ETA rule in reducing the occurrence of long waiting times.

It should be noted that our model explicitly separates the training and deployment phases. Although training requires time for model optimization, deployment is highly efficient: at each decision point, the controller performs only a single forward pass. In practice, this results in lightweight inference that can be executed on standard edge hardware while comfortably satisfying the event-driven latency constraints of elevator dispatching. Although the average reduction in waiting time is only on the order of a few seconds, the impact is meaningful: with thousands of passengers served each day, even small per-passenger improvements accumulate into substantial aggregate time savings and noticeably enhance overall service experience. More importantly, we consistently observe significant reductions in long waiting times. Mitigating this tail risk further yields improvements in service quality that typically outweigh the modest additional maintenance and computational costs required to deploy a learned policy.

Table 4. Long Waiting Time Comparison under Four Traffic Patterns

		UpPeak	InterFloor	LunchPeak	DownPeak
ETA	$p_{>60}$	0.377	0.072	0.555	0.059
	$WT_{0.95}$	446.98	67.86	292.26	63.19
	$WT_{0.75}$	119.43	32.88	140.74	30.81
IL + RL (w.o. perfect info)	$p_{>60}$	0.367	0.058	0.509	0.047
	$WT_{0.95}$	380.62	62.84	278.592	58.96
	$WT_{0.75}$	104.01	32.21	116.589	29.78
IL + RL (w. perfect info)	$p_{>60}$	0.352	0.055	0.494	0.045
	$WT_{0.95}$	356.25	61.46	270.97	58.16
	$WT_{0.75}$	98.44	32.03	111.24	29.79

$p_{>60}$: Percentage of passengers waiting longer than 60 seconds.

$WT_{0.95}$: 0.95 percentile of waiting time.

$WT_{0.75}$: 0.75 percentile of waiting time.

5.2.4 Effect of Using Perfect Information. In addition to our previous experiments, we conducted tests assuming the availability of perfect information, which provided precise knowledge of the number of passengers waiting behind hall calls and the number of passengers exiting the elevator car on each floor. As shown in Tables 3 and 4, our model's performance with perfect information was slightly better compared to the scenario without such information, as expected. An interesting observation is that the pre-trained policy under the DownPeak traffic pattern could outperform the ETA rule. Furthermore, the utilization of perfect information appeared to have a more substantial impact on performance for the UpPeak and LunchPeak traffic patterns compared to the Interfloor and DownPeak patterns. This discrepancy can be attributed to the higher traffic volumes in the UpPeak and LunchPeak scenarios, where the detailed information proves to be more valuable in optimizing dispatching decisions. Conversely, the Interfloor and DownPeak traffic patterns exhibit relatively lighter traffic volumes, resulting in the number of passengers waiting behind the hall calls and exiting the elevator car being closer to 1, resembling the previous scenario without perfect information.

6 Conclusion

In this article, we tackled the elevator dispatching problem by formulating it as a SMDP and proposed a two-phase model that combines IL and RL to effectively solve the formulated SMDP. Additionally, we introduced a novel "direct-effect" update interval, which significantly improved the efficiency of RL training. The empirical results confirmed the advantages of incorporating both IL and RL for elevator dispatching, as well as the effectiveness of the proposed "direct-effect" update interval. Furthermore, we explored the benefits of utilizing perfect information for dispatching decisions, and our model outperformed four heuristic rules across various traffic patterns in terms of average waiting time and the percentage of long waiting times.

While deep RL has gained popularity in various industrial domains, its direct applicability to complex and dynamic problems, such as elevator dispatching, may be limited. The approach of leveraging IL as an initial step to transfer existing industrial expert knowledge and then refining the pre-trained policy via RL appears to be a more promising strategy for solving such problems. Moreover, we believe that the concept of the proposed "direct-effect" update interval could be applied to other multi-resource scheduling problems (e.g., AGVs, machines), potentially yielding performance improvement. For example, in a job shop scheduling environment, the direct

effect of assigning a job to a particular machine may last until the machine finishes processing the job, rather than being limited to the next decision moment of assigning other jobs to other machines.

In the current implementation, the system adopts a static assignment strategy: once a hall call assignment decision is made, it will not change. This approach is simple and computationally efficient, but it may limit performance compared to fully dynamic reassignment mechanisms, in which both newly released and previously assigned requests must be jointly optimized within a single decision event. Such dynamic reassignment requires making multiple allocation decisions simultaneously, substantially increasing the complexity of the problem formulation as well as the design of the decision-making network. We are actively extending our framework to address this more challenging setting.

References

- [1] Pieter Abbeel and Andrew Y. Ng. 2011. Inverse reinforcement learning. In *Encyclopedia of Machine Learning*. Claude Sammut and Geoffrey I. Webb (Eds.), Springer, 554–558.
- [2] Saurabh Arora and Prashant Doshi. 2021. A survey of inverse reinforcement learning: Challenges, methods and progress. *Artificial Intelligence* 297 (2021), 103500.
- [3] Gina Barney and Lutfi Al-Sharif. 2015. *Elevator Traffic Handbook: Theory and Practice*. Routledge.
- [4] Jianguo Chen, Kenli Li, Keqin Li, Philip S. Yu, and Zeng Zeng. 2021. Dynamic bicycle dispatching of dockless public bicycle-sharing systems using multi-objective reinforcement learning. *ACM Transactions on Cyber-Physical Systems* 5, 4 (2021), 1–24.
- [5] Pablo Cortés, Juan Larrañeta, and Luis Onieva. 2004. Genetic algorithm for controllers in elevator groups: Analysis and simulation during lunchpeak traffic. *Applied Soft Computing* 4, 2 (2004), 159–174.
- [6] Pablo Cortes, Jesus Munuzuri, Alejandro Vazquez-Ledesma, and Luis Onieva. 2021. Double deck elevator group control systems using evolutionary algorithms: Interfloor and lunchpeak traffic analysis. *Computers & Industrial Engineering* 155 (2021), 107190.
- [7] Robert H. Crites and Andrew G. Barto. 1998. Elevator group control using multiple reinforcement learning agents. *Machine Learning* 33, 2 (1998), 235–262.
- [8] Muna Hamdi and D. J. Mulvaney. 2007. Prioritised A* search in real-time elevator dispatching. *Control Engineering Practice* 15, 2 (2007), 219–230.
- [9] Tapio Hautamäki. 2021. Multiobjective Optimization Model for Elevator Call Allocation. Retrieved from https://sal.aalto.fi/publications/pdf-files/theses/mas/thau21_public.pdf
- [10] Jonathan Ho and Stefano Ermon. 2016. Generative adversarial imitation learning. In *Proceedings of the 30th International Conference on Neural Information Processing Systems (NIPS '16)*, 4572–4580.
- [11] Hao Hu, Xiaoliang Jia, Qixuan He, Shifeng Fu, and Kuo Liu. 2020. Deep reinforcement learning based AGVs real-time scheduling with mixed rule for flexible shop floor in industry 4.0. *Computers & Industrial Engineering* 149 (2020), 106749.
- [12] Ahmed Hussein, Mohamed Medhat Gaber, Eyad Elyan, and Chrisina Jayne. 2017. Imitation learning: A survey of learning methods. *ACM Computing Surveys* 50, 2 (2017), 1–35.
- [13] Masaki Ikuta, Kenichi Takahashi, and Michimasa Inaba. 2013. Strategy selection by reinforcement learning for multi-car elevator systems. In *Proceedings of the 2013 IEEE International Conference on Systems, Man, and Cybernetics*. IEEE, 2479–2484.
- [14] Anton Jansson and Kristoffer Uggle Lingvall. 2015. Elevator Control Using Reinforcement Learning to Select Strategy. Retrieved from https://www.kth.se/social/files/588617c2f276547fe1dbf8d2/AJanssonKUggleLingvall_dkand15.pdf
- [15] Jana Koehler. 2001. From theory to practice: AI planning for high performance elevator control. In *Proceedings of the Joint German/Austrian Conference on AI, Advances in Artificial Intelligence (KI '01)*. Springer, 459–462.
- [16] Jana Koehler and Daniel Ottiger. 2002. An AI-based approach to destination control in elevators. *AI Magazine* 23, 3 (2002), 59–59.
- [17] Enming Liang, Kexin Wen, William H. K. Lam, Agachai Sumalee, and Renxin Zhong. 2022. An integrated reinforcement learning and centralized programming approach for online taxi dispatching. *IEEE Transactions on Neural Networks and Learning Systems* 33, 9 (2022), 4742–4756.
- [18] Erfan Farhangi Maleki, Dixit Bhatta, and Lena Mashayekhy. 2023. A game-theoretic approach to energy-efficient elevator scheduling in smart buildings. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 53, 7 (2023), 3944–3955.

- [19] Srikumar Ramalingam, Arvind Raghunathan, and Daniel Nikovski. 2017. Submodular function maximization for group elevator scheduling. *Proceedings of the International Conference on Automated Planning and Scheduling* 27, 1 (2017), 233–241.
- [20] Aiyong Rong, Henri Hakonen, and Risto Lahdelma. 2003. *Estimated Time of Arrival (ETA) Based Elevator Group Control Algorithm with More Accurate Estimation*. Turku Centre for Computer Science.
- [21] Farli Rossi, Jaka P. Sembiring, Akhmad Jayadi, Novia U. Putri, and Prasetyo Nugroho. 2021. Implementation of fuzzy logic in PLC for three-story elevator control system. In *Proceedings of the 2021 International Conference on Computer Science, Information Technology, and Electrical Engineering (ICOMITEE)*, 179–185. DOI: <https://doi.org/10.1109/ICOMITEE53461.2021.9650221>
- [22] Mirko Ruokokoski, Janne Sorsa, Marja-Liisa Siikonen, and Harri Ehtamo. 2016. Assignment formulation for the elevator dispatching problem with destination control and its performance analysis. *European Journal of Operational Research* 252, 2 (2016), 397–406.
- [23] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. 2015. Trust region policy optimization. In *Proceedings of the International Conference on Machine Learning*. PMLR, 1889–1897.
- [24] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. 2015. High-dimensional continuous control using generalized advantage estimation. arXiv:1506.02438. Retrieved from <https://arxiv.org/abs/1506.02438>
- [25] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. arXiv:1707.06347. Retrieved from <https://arxiv.org/abs/1707.06347>
- [26] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. MIT Press.
- [27] Emre Oner Tartan and Cebirali Ciftlikli. 2016. A genetic algorithm based elevator dispatching method for waiting time optimization. *IFAC-PapersOnLine* 49, 3 (2016), 424–429.
- [28] Faraz Torabi, Garrett Warnell, and Peter Stone. 2018. Behavioral cloning from observation. arXiv:1805.01954. Retrieved from <https://arxiv.org/abs/1805.01954>
- [29] Alex Valdivielso and Toshiyuki Miyamoto. 2010. Multicar-elevator group control algorithm for interference prevention and optimal call allocation. *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans* 41, 2 (2010), 311–322.
- [30] Eugene Vinitzky, Nathan Lichtlé, Kanaad Parvate, and Alexandre Bayen. 2023. Optimizing mixed autonomy traffic flow with decentralized autonomous vehicles and multi-agent reinforcement learning. *ACM Transactions on Cyber-Physical Systems* 7, 2 (2023), 1–22.
- [31] Jiansong Wan and Hayong Shin. 2022. Predictive vehicle dispatching method for overhead hoist transport systems in semiconductor fabs. *International Journal of Production Research* 60, 10 (2022), 3063–3077.
- [32] Ruihang Wang, Zhiwei Cao, Xin Zhou, Yonggang Wen, and Rui Tan. 2022. Green data center cooling control via physics-guided safe reinforcement learning. *ACM Transactions on Cyber-Physical Systems* 8, 2 (2022), 1–26.
- [33] Shu Wang, Xuejian Gong, Mulang Song, Cindy Y. Fei, Stefan Quaadgras, Jianyuan Peng, Pan Zou, Jerred Chen, Wei Zhang, and Roger J. Jiao. 2021. Smart dispatching and optimal elevator group control through real-time occupancy-aware deep learning of usage patterns. *Advanced Engineering Informatics* 48 (2021), 101286.
- [34] Qinglai Wei, Lingxiao Wang, Yu Liu, and Marios M. Polycarpou. 2020. Optimal elevator group control via deep asynchronous actor–critic learning. *IEEE Transactions on Neural Networks and Learning Systems* 31, 12 (2020), 5245–5256.
- [35] Jing Zhang, Athanasios Tsiligkaridis, Hiroshi Taguchi, Arvind Raghunathan, and Daniel Nikovski. 2022. Transformer networks for predictive group elevator control. In *2022 European Control Conference (ECC)*. IEEE, 1429–1435.
- [36] Boyuan Zheng, Sunny Verma, Jianlong Zhou, Ivor W. Tsang, and Fang Chen. 2022. Imitation learning: Progress, taxonomies and challenges. *IEEE Transactions on Neural Networks and Learning Systems* 35, 5 (2022), 6322–6337.

Appendices

A Comparison between SMDP and MDP Formulation

We conducted an ablation study in which we modified our framework to use immediate rewards (i.e., without integration over intervals) and applied a constant discount factor, thereby replicating a traditional MDP setting. We then compared the training performance of this MDP variant with our proposed SMDP-based approach. As shown in Figure A1, the training process using the SMDP-based model significantly outperforms that of the MDP variant. Notably, the MDP-based model even diverges during training. We believe this is because the MDP formulation applies a fixed discount factor regardless of the time interval, which can vary significantly. This mismatch likely confuses the critic network during training, resulting in inaccurate advantage estimates and ultimately degrading the performance of the actor network.

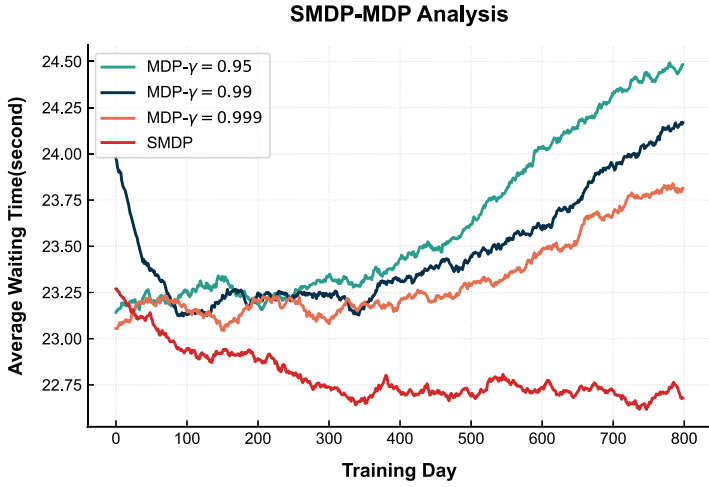


Fig. A1. Training process of SMDP and MDP formulations.

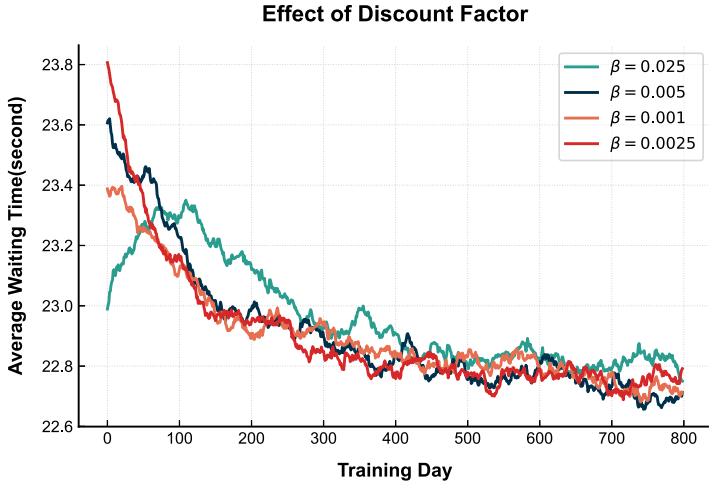


Fig. B1. Training process under various discount factor settings.

B Effects of Representative Hyperparameters

Discount Factor. We first investigated the effect of the hyperparameter β in reward definition Equation (1) and loss calculation Equations (5) and (6), which controls the discount factor. In our formulation, given the range of time intervals, increasing β results in a greater discount, causing the agent to focus more on immediate rewards and less on future outcomes. This simplification makes value estimation easier but comes at the cost of reducing the agent's ability to make far-sighted decisions. In contrast, a smaller β results in less discounting, encouraging the agent to consider the long-term effects of its actions. While this leads to more far-sighted decisions, it also increases the difficulty of accurate value estimation.

We trained our models under the DownPeak traffic pattern using different β values. As shown in Figure B1, the training process with $\beta = 0.025$ demonstrates relatively poor performance, as it

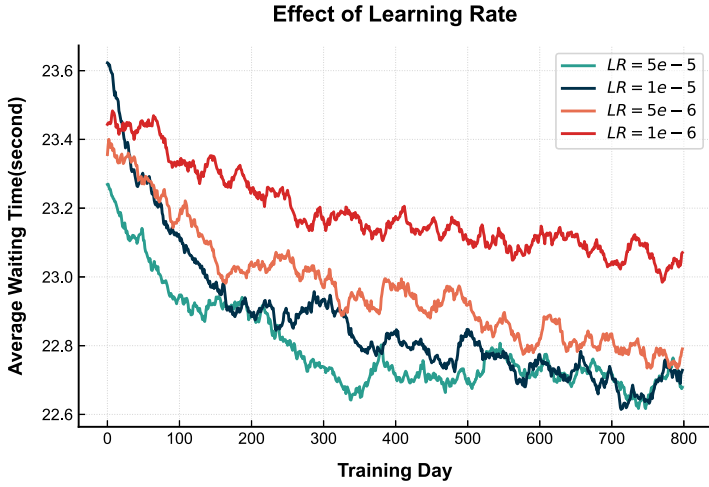


Fig. B2. Training process under various clip ratio settings.

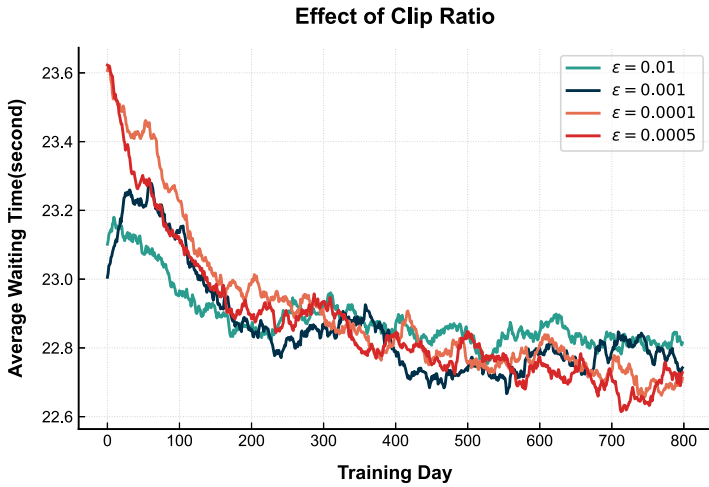


Fig. B3. Training process under various learning rate settings.

discounts too heavily and fails to account for long-term effects. In contrast, training with smaller β values yield better performance. Among these settings, $\beta = 0.005$ achieves the best performance.

Learning Rate. We trained our model using various learning rates to evaluate their impact on the training process. As shown in Figure B2, the learning rate significantly influences both convergence speed and training stability. Smaller learning rates result in more stable training but converge more slowly. Among the configurations tested, a learning rate of 5×10^{-5} achieved the best overall performance, offering the fastest convergence without sacrificing too much stability.

Clip Ratio. We also investigated the effect of the clipping ratio ϵ in Equation (4), which is a critical hyperparameter in the Proximal Policy Optimization (PPO) algorithm. As shown in Figure B3, training with larger ϵ values (e.g., 0.1, 0.001) results in faster convergence during the early stages, as larger ϵ permits more substantial policy updates. However, in the later stages, these settings fail to converge to higher-quality policies due to instability introduced by overly aggressive updates.

In contrast, smaller ϵ values (e.g., 0.0001, 0.0005) lead to more stable training and ultimately better convergence, despite slower progress in the initial phase. Based on this analysis, we set $\epsilon = 0.0005$ for all experiments, as it achieves a good balance between stability and learning speed.

Received 5 January 2024; revised 14 November 2025; accepted 8 January 2026